

June 9, 2026

First live run against deployed dev · 12 minutes · \$0.00 spend

TODAY'S SNAPSHOT

The harness found three real authorization problems in the deployed app. Most of the rest of the noise was the test tool, not the app.

The deployed API treats whatever JWT arrives in the Authorization header as authoritative without verifying the signature. As a result, any signed-in user can claim to be a CISO and the backend lets them in. We saw the same pattern three ways — forged group claims, valid lower-privilege tokens, and access tokens used in place of ID tokens.

Of the 91 “failures” the harness recorded, **19 are real and need a backend fix**. The other 72 are issues in the test harness itself that we'll clean up before the next run.

The headlines

SEVERITY	WHAT
CRITICAL	Any signed-in user can forge a token claiming CISO group membership and the API accepts it. Signature verification is off on protected routes.
HIGH	SOC, GRC, and Employee tokens reach CISO-only routes and get back 200 OK. The backend doesn't check the group.
HIGH	The API accepts Cognito access tokens where it should require ID tokens. 24 routes affected. Less acute, same root family.
LOW	One transient HTTP 502 from the chat endpoint mid-run. Single event. Watch for repeats.

What worked. Cost stayed at \$0.00 against a \$1.00 cap, preflight was all green, all four demo personas authenticated against real Cognito, and the harness's probe-budget controls fired exactly when expected. The findings above were the harness doing its job.

What needs fixing in the app

CRITICAL Forged CISO claims grant unauthorized access

Anyone with a legitimate sign-in (SOC, GRC, Employee) can rewrite the `cognito:groups` claim inside their token to `["ciso"]` and hit the three CISO-only endpoints — token usage, token usage summary, and action approval. All ten of our probes returned 200 OK. The expected response is 401 or 403.

The root cause is almost certainly missing JWT signature verification. The attacker doesn't have the Cognito signing key, so an accepted rewritten token means the server isn't checking the signature.

IMPACT An employee can today see all four personas' token spend and approve compliance changes that auto-roll updates to AWS Config and Zscaler.

FIX DIRECTION Verify JWT signatures wherever a CISO claim is trusted, and re-derive the user's group from Cognito rather than reading it from the token. Either ensure the API Gateway Cognito authorizer is attached to every protected route, or validate the JWT against the Cognito public keys inside the Lambda handler.

HIGH Cross-persona authorization isn't enforced

Without any forgery — just using a real SOC, GRC, or Employee token — requests against the CISO-only routes still come back 200. The frontend hides these pages from the wrong personas, but the backend trusts whoever has a valid token.

FIX DIRECTION Audit every route handler in `api_handler.py` and confirm the CISO gate is being called before any business logic. A small decorator pattern (something like `@require_group("ciso")`) would make this hard to forget when new routes are added.

HIGH API accepts access tokens where ID tokens are required

Cognito issues two kinds of tokens at sign-in: an ID token (carries identity claims like email) and an access token (carries scopes for API authorization). Most of our endpoints should require the ID token. Right now they accept either. 24 routes are affected.

WHY IT MATTERS Less urgent than the two above — both tokens are still signed by Cognito — but access tokens lack identity claims, so any handler relying on email or username will misbehave silently when the wrong token type comes in.

FIX DIRECTION One-line check in the token decoder: `assert token_use == "id"` and reject otherwise.

What we need to clean up in the test tool

These produced most of the noise in today's run. None of them are bugs in the deployed application — they're fixes in the test harness so tomorrow's report is a cleaner verdict.

HIGH Browser tests never managed to sign in

All 60 page tests landed on the sign-in screen instead of the page they asked for. The harness writes the Cognito tokens into the browser the way the app should expect, but the deployed React app isn't accepting them and bounces the user back to sign in.

This is the single largest source of red in today's report — and it has nothing to do with the deployed app being broken. We never got the browser logged in, so every page test failed at the "you're not signed in" gate.

FIX DIRECTION Half a day of diagnosis. Either the token shape we write doesn't match what the React code expects, or there's a downstream validation step we're missing. Worst case, we switch the harness to log in through the real Cognito hosted sign-in page — slower per run but bulletproof.

MEDIUM LLM safety check misread clean refusals as ambiguous

We sent 30 jailbreak prompts to the chat endpoint. The model actually refused 26 of them cleanly with the standard message *"Your request was blocked by content safety policies"* — but our test tool was looking for older refusal phrases like "I cannot" and missed it. Five-minute fix.

BOTTOM LINE The safety filter is working. Today's report shows zero LLM failures, but the "skipped" numbers will become "passed" after the fix.

MEDIUM API fuzz layer ran out of time

The API fuzz layer sends thousands of malformed inputs at every route. At the polite 5-requests-per-second throttle it needs about 19 minutes; we give it 10. The layer simply ran out of clock and never wrote its results.

FIX DIRECTION Cut the fuzz layer down to a single persona — the persona-gating layer is already covering that surface — and bump the per-layer time limit to half an hour. About 30 minutes of work.

MEDIUM The final report didn't render

All four layers ran and wrote their data. The very last step — rendering the polished HTML report — refused to build because one entry pointed at an evidence file that didn't exist (a casualty of the sign-in issue above). The check was too strict and killed the whole report.

FIX DIRECTION Change the check from a hard failure to a soft warning. The findings table will mark the missing evidence with a flag instead of aborting the build. Fifteen-minute fix.

LOW Cost-DoS check couldn't read its measurement

One of the LLM probes asks for an exhaustive 2,000-word response and then checks that the runtime capped the actual output. The chat endpoint doesn't report the token count back in its response, so the check skipped instead of running. Backend change needed to surface that number on the response payload — about an hour of work and useful beyond this one test.

Action plan

DAY 1 — URGENT Patch the backend authorization

Add JWT signature verification, audit the CISO gate on every route handler, and assert the token type is an ID token in the decoder. Add a regression test so this can't come back. Redeploy through CodePipeline, re-run the harness, and the 19 high-severity rows should flip to green.

DAY 1 — IN PARALLEL Tidy the test harness

Soften the report builder's evidence check, add the modern refusal phrases to the LLM classifier, and cut the fuzz layer down to one persona. Roughly an hour total. Most of today's noise disappears.

DAY 2 Get the browser tests signing in

Diagnose the storage-state mismatch and pick a fix path. Once this lands the 65 false E2E failures collapse to whatever the real number is — which I expect to be much smaller.

DAY 3 Polish and lock in a baseline

Surface token usage on the chat response so the cost-DoS check can run, then promote a clean run as the green baseline. From that point on, every daily report shows a diff against the previous green — regressions and improvements both become obvious at a glance.